

Forma K — Sistema de Gestión de Cursos y Matrículas

Informe de Unidad 1 · Integración de Sistemas

Universidad de Concepción

Facultad de Ingeniería

Grupo: [completar]

Integrantes: [completar]

Fecha: [completar]

1. Análisis del problema

AprendeMás opera dos sistemas que evolucionaron por separado. Cupos conoce la oferta de cursos y la capacidad restante; Matrículas guarda estudiantes y sus inscripciones. Sin integración, una matrícula puede exceder la capacidad de una sala.

La necesidad es que una matrícula solo se confirme si Cupos la autoriza. Las restricciones relevantes son: API pública versionada, contrato formal, servicios y bases independientes, autenticación y comportamiento seguro ante una dependencia caída.

Se construyó una solución deliberadamente acotada: no intenta introducir una transacción distribuida. En cambio, la decisión de ocupar se mantiene atómica dentro de Cupos y la API informa con precisión los errores recuperables.

2. Diseño y arquitectura

Matrículas recibe HTTP REST bajo /v1. Mantiene estudiantes y matrículas en SQLite. Para verificar disponibilidad usa un canal gRPC reutilizable contra Cupos, que posee una segunda base SQLite. Redis es infraestructura compartida de caché e idempotencia, no fuente de verdad académica.

Cliente HTTP → Matrículas (REST) → Cupos (gRPC/protobuf)



Redis (caché e idempotencia)

Esta estructura reduce el acoplamiento de datos: Matrículas persiste el identificador del curso, pero nunca una copia administrable de sus cupos. Docker Compose declara los tres contenedores y variables de conexión.

3. ADR-001: frontera de servicios

Contexto: Cupos y Matrículas tienen propietarios y datos con sensibilidad diferente. Alternativas: monolito y base común; procesos separados con base común; dos servicios con base por servicio.

Decisión: se eligieron dos servicios con bases separadas. Cupos posee Curso y sus contadores; Matrículas posee Estudiante y Matrícula. La frontera evita que un cambio al esquema de cupos afecte al historial de matrículas.

Costo aceptado: la operación cruza la red y puede requerir conciliación si falla tras ocupar un cupo. Producción evolucionaría hacia reserva identificada y proceso de compensación, no hacia acceso cruzado a datos.

4. ADR-002: REST y gRPC

Alternativas: REST/JSON para todo, gRPC para todo, o REST externo con gRPC interno. Se eligió la última opción: REST permite a portal y personal usar verbos, códigos y JSON; gRPC genera stubs tipados y ofrece streaming para el catálogo.

La documentación de gRPC indica que un canal usa conexiones HTTP/2 y streams concurrentes [1]. El experimento local no pretende generalizar rendimiento de red, pero sí midió que el Curso de prueba ocupa 39 bytes en protobuf contra 94 bytes JSON. Esa diferencia apoya usar protobuf en el tramo interno repetido.

Costo: hay dos contratos que mantener. Se mitiga con `cupos.proto` y `openapi.yaml` versionados y con pruebas automáticas de su estructura.

5. ADR-003: contrato y evolución

REST usa /v1; cambios incompatibles se publican en /v2 con una ventana de deprecación comunicada. Agregar un atributo opcional es compatible, pero modificar semántica o eliminar uno requerido requiere versión nueva.

En protobuf los campos reciben números estables. La guía oficial señala que cambiar números existentes no es seguro; un campo nuevo con tag no usado sí permite que clientes antiguos ignoren lo desconocido [2]. Por tanto, una futura descripción = 5 es compatible, pero reenumerar cupos_disponibles no lo es.

Costo: coexistirán versiones durante migración y el equipo debe conocer consumidores antes de retirar una ruta o campo.

6. ADR-004: resiliencia

Sin timeout, una dependencia lenta puede consumir los recursos de Matrículas. Con disponibilidad basada solo en caché se podría sobrepasar la sala. Se eligió fail-fast: deadline gRPC de 2 segundos, 503 para UNAVAILABLE y 504 para DEADLINE_EXCEEDED.

Los errores usan application/problem+json con type, title, status y detail. RFC 9457 estandariza esta representación para que clientes HTTP reciban detalle accionable sin una traza de implementación [3].

Costo: se rechaza temporalmente una matrícula que podría completar más tarde. El cliente debe reintentar con Idempotency-Key; la demo debe detener Cupos y observar el 503.

7. Experimento: tamaño y serialización

Hipótesis: la representación protobuf del mismo Curso utiliza menos bytes y menor tiempo de serialización que JSON sin compresión. Variables controladas: mismo objeto, mismo proceso Python 3.14, diez series y 10.000 serializaciones por serie. Se midió len de la salida y perf_counter_ns.

Formato	Tamaño	Media (ns/op)	Desv. estándar
Protobuf	39 B	238	72
JSON compacto	94 B	3.770	273

Resultado: protobuf fue 58,5 % menor por mensaje y más rápido en este microbenchmark. Límite: no mide red, compresión gzip, deserialización, concurrencia ni rendimiento de otro lenguaje. El script docs/experimento_serializacion.py permite repetir exactamente el procedimiento.

8. Contexto: privacidad y costo

El factor elegido es privacidad de datos estudiantiles. Separar la base de Matrículas limita la exposición de nombres y correos al servicio de Cupos, que no los necesita. Los errores no incluyen stack traces y el acceso público exige Bearer token.

La elección tiene costo operacional: tres contenedores y Redis. Se justifica porque Redis además contiene las claves de idempotencia y reduce consultas repetidas. El patrón cache-aside consulta la caché primero, carga la fuente ante miss y elimina la clave después de una escritura; Redis documenta TTL e invalidación explícita para limitar datos obsoletos [4].

Para producción se reemplazaría el token de desarrollo por un emisor JWT, secretos gestionados, TLS entre servicios, rotación y auditoría de acceso.

9. Ejecución, verificación y reflexión

Ejecución: `docker compose up --build`. La API está en `http://localhost:8000` y Swagger en `/docs`. El README contiene solicitudes PowerShell para crear estudiante, matricular, consultar y revertir. Para la falla: `docker compose stop cupos` y repetir una matrícula; se espera 503 `problem+json`.

Verificación automatizada: cuatro pruebas pasan. Dos prueban curso inexistente y el ciclo ocupar/liberar; dos verifican RPC, streaming, OpenAPI, Bearer y `problem+json`. La validación final de contenedores debe realizarse con Docker Desktop ejecutándose.

Reflexión: la solución evita sobreconstrucción, pero aún falta una reserva durable para el fallo entre servicios, observabilidad con métricas/trazas y pruebas de integración reales en contenedores. Son extensiones justificadas una vez que los requisitos obligatorios estén demostrados.

10. Referencias

[1] gRPC. Performance Best Practices. <https://grpc.io/docs/guides/performance/> (consultado 2026-09-22).

[2] Protocol Buffers. Language Guide (proto3), Updating a Message Type.
<https://protobuf.dev/programming-guides/proto3/> (consultado 2026-09-22).

[3] IETF. RFC 9457: Problem Details for HTTP APIs. <https://www.rfc-editor.org/rfc/rfc9457.html> (2023).

[4] Redis. Cache-aside. <https://redis.io/docs/latest/develop/use-cases/cache-aside/> (consultado 2026-09-22).

Anexos del repositorio: openapi.yaml, cupos.proto, cuatro ADR, pruebas y guion de video.